Abstract

Abstract I

This paper describes a small, tested **domain language for specifying controlled methods** — the **methods-paper exemplar** of the Research Project Template. Unlike a results paper, this manuscript's subject is the methodology itself: a controlled vocabulary, a unit system with dimensional safety, four staged validation gates, and a deterministic compiler, implemented in `projects/templates/template_methods_paper/src/methods_dsl/` and described section by section in [@sec:methodology]. The domain language's vocabulary is informed by BPL (Biology Programming Language, [@bpl2026]), an upstream reference that encodes laboratory protocols as programs with biology-native types, staged validation, and deterministic compilation; this exemplar generalizes BPL's intent vocabulary and pipeline shape from wet-lab protocols to any controlled procedure.

Abstract II

A `Method` is a name, a set of typed parameters and resources, and an ordered, dependent set of steps — constructed directly as frozen Python dataclasses (`src/methods_dsl/model.py`) rather than parsed from new text syntax. Every `Quantity` carries a unit that resolves to one of 18 controlled units across seven dimensions, and every step names one of 9 controlled-vocabulary intents (`src/methods_dsl/vocabulary.py`), executable on one of 3 backends. 4 staged gates — structural, semantic, plan, and target — validate a method before `compile_method` (`src/methods_dsl/compiler.py`) deterministically schedules it with Kahn's algorithm [@kahn1962topological] and hashes the canonical plan with SHA-256.

Abstract III

We demonstrate the language on 2 worked example methods spanning both domains BPL's design targets and the domains it generalizes to: a manual wet-lab preparation (`PBSPreparation`, 5 steps, target `human`, plan hash `313b9b17de98`) and an automated instrument-calibration procedure (`SensorCalibrationSweep`, 4 steps, target `automated`, plan hash `d89cced19be6`). Live re-compilation determinism check: **Yes**. Across both methods, 8 of 8 staged-gate evaluations pass. A demonstration provenance hash-chain (`src/methods_dsl/trust.py`) of length 3 verifies as **Yes**.

Abstract IV

Contributions are **methodological** and **architectural**. On the methods side, we show that a controlled vocabulary expressed as typed dataclasses — not a parsed grammar — is sufficient to reproduce BPL's core safety properties (dimensional safety, staged validation, deterministic compilation) at a scope appropriate for a template exemplar. On the architecture side, the DSL is exercised by a zero-mock test suite under the repository's configured project coverage gate, generates 14 artifacts (1 figures, 7 data files, 6 reports) per pipeline run, and injects reproducibility metadata (configuration hash `a0f000565bef6a79`, build timestamp `2026-07-12T22:24:06Z`) into [@sec:reproducibility].

Keywords: methods paper, domain-specific language, controlled methods, deterministic compilation, staged validation, dimensional analysis